

EX. NO: 6

IMPLEMENTATION OF RAIL FENCE CIPHER

AIM:

To write a C program to implement the Rail Fence transposition technique.

ALGORITHM:

1. Start the program.
2. Enter the plain text.
3. Remove the spaces from the plain text.
4. Enter the number of rails.
5. Arrange the characters of the plain text in a zigzag pattern across the given number of rails.
6. Read the characters row by row.
7. Display the encrypted text.
8. Stop the program.

PROGRAM:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void main()
```

```
{
```

```
    char plain[100], text[100];
```

```
    char rail[10][100];
```

```
    int key, i, j, row = 0;
```

```
    int direction = 1;
```

```
int len, k = 0;
```

```
printf("Enter the plain text: ");
```

```
fgets(plain, sizeof(plain), stdin);
```

```
/* Remove spaces and newline */
```

```
for(i = 0; plain[i] != '\0'; i++)
```

```
{
```

```
    if(plain[i] != ' ' && plain[i] != '\n')
```

```
    {
```

```
        text[k] = plain[i];
```

```
        k++;
```

```
    }
```

```
}
```

```
text[k] = '\0';
```

```
printf("Enter the number of rails: ");
```

```
scanf("%d", &key);
```

```
len = strlen(text);
```

```
/* Initialize the rail matrix */
```

```
for(i = 0; i < key; i++)
```

```
{
```

```
    for(j = 0; j < len; j++)
```

```
    {
```

```
        rail[i][j] = '\0';
```

```
    }
```

```
}
```

```
/* Arrange text in zigzag pattern */
```

```
for(i = 0; i < len; i++)
```

```
{
```

```
    rail[row][i] = text[i];
```

```
    if(row == key - 1)
```

```
        direction = -1;
```

```
    else if(row == 0)
```

```
direction = 1;
```

```
row = row + direction;
```

```
}
```

```
printf("\nPLAIN TEXT: %s", plain);
```

```
printf("ENCRYPTED TEXT: ");
```

```
/* Read the rails row by row */
```

```
for(i = 0; i < key; i++)
```

```
{
```

```
    for(j = 0; j < len; j++)
```

```
    {
```

```
        if(rail[i][j] != '\0')
```

```
        {
```

```
            printf("%c", rail[i][j]);
```

```
        }
```

```
    }
```

```
}
```

}

OUTPUT:

```
main.c
55 printf("\nPLAIN TEXT: %s", plain);
56
57 printf("ENCRYPTED TEXT: ");
58
59 /* Read the rails row by row */
60 for(i = 0; i < key; i++)
61 {
62     for(j = 0; j < len; j++)
63     {
64         if(rail[i][j] != '\0')
65         {
66             printf("%c", rail[i][j]);
67         }
68     }
69 }
70 }
```

Output

Enter the plain text: PEEK A BOO
Enter the number of rails: 2

PLAIN TEXT: PEEK A BOO
ENCRYPTED TEXT: PEAOEKBO

=== Code Exited With Errors ===

```
main.c
55 printf("\nPLAIN TEXT: %s", plain);
56
57 printf("ENCRYPTED TEXT: ");
58
59 /* Read the rails row by row */
60 for(i = 0; i < key; i++)
61 {
62     for(j = 0; j < len; j++)
63     {
64         if(rail[i][j] != '\0')
65         {
66             printf("%c", rail[i][j]);
67         }
68     }
69 }
70 }
```

Output

Enter the plain text: GO TO THE PARTY
Enter the number of rails: 3

PLAIN TEXT: GO TO THE PARTY
ENCRYPTED TEXT: GTAOOHPRYTET

=== Code Exited With Errors ===

RESULT:

Thus, the Rail Fence transposition technique was successfully implemented using C language, and the given plaintext was successfully encrypted using the specified number of rails.